

T0.2 — Stripe webhook signature obligatorio

Diagnostico de solo lectura · Fecha: 2026-05-01 · Modo: solo lectura. · Sin cambios, sin commits, sin push.

Reporte de revision previa al PR para hacer obligatoria la verificacion de firma de Stripe en el endpoint /webhook/stripe en produccion. Cierra el riesgo P0 de acreditacion forjable de creditos.

1. Diagnostico actual

1.1 Flujo Stripe webhook hoy

```
Stripe -> POST https:///webhook/stripe
|
agent/main.py:2112 webhook_stripe(request)
| lee body + header 'stripe-signature'
v
agent/billing.py:366 verificar_firma_stripe(body, sig)
| hay STRIPE_WEBHOOK_SECRET?
+-- NO -> json.loads(body) -> retorna evento 'verificado' (warning) [INSEGURO]
+-- SI -> stripe.Webhook.construct_event(body, sig, secret)
|
|   +-- firma valida -> retorna evento
|   +-- firma invalida -> retorna None
|
v
Si evento es None -> 400 'signature_invalid'
Si evento OK -> procesar_evento_stripe(evento)
v
checkout.session.completed -> acreditar(telefono, creditos, session_id)
v
Notificacion WhatsApp al usuario 'Gracias por tu compra'
```

1.2 Comportamiento actual sin STRIPE_WEBHOOK_SECRET

agent/billing.py:371-377:

```
secret = webhook_secret or os.getenv("STRIPE_WEBHOOK_SECRET", "")
if not secret:
    logger.warning("[BILLING] STRIPE_WEBHOOK_SECRET no configurada – aceptando sin verificar (INSEGURO)")
    try:
        return json.loads(payload.decode("utf-8"))
    except Exception:
        return None
```

Riesgo P0 confirmado. Un atacante que conoce la URL /webhook/stripe puede:

- Enviar un POST con JSON forjado: `{'type': 'checkout.session.completed', 'data': {'object': {'id': 'atk_001', 'client_reference_id': '<telefono_victima>', 'metadata': {'telefono': '<tel>', 'creditos': '10000'}}}}`
- El handler acredita 10000 creditos al telefono atacante.
- El atacante puede generar imagen/video/voz sin pagar nunca.
- stripe_session_id actua como dedupe, pero el atacante puede generar IDs arbitrarios para acreditar repetidamente.

1.3 Usos de STRIPE_WEBHOOK_SECRET en el repo

Archivo	Linea	Uso
agent/billing.py	371	os.getenv('STRIPE_WEBHOOK_SECRET', '') con fallback inseguro
agent/billing.py	373	log warning
landing/app/api/webhook/route.ts	20	process.env.STRIPE_WEBHOOK_SECRET! (non-null assertion). Si falta, lanza error en runtime al primer webhook. Independiente del backend Python.
tests/test_billing.py	197, 204	dos tests TestVerificarFirma que verifican el comportamiento permisivo actual

1.4 Tests existentes que cambiaran de comportamiento

tests/test_billing.py:195-207:

```
class TestVerificarFirma:
    def test_sin_secret_configurado_acepta_payload(self, monkeypatch):
        monkeypatch.delenv("STRIPE_WEBHOOK_SECRET", raising=False)
        ...
        ev = agent.billing.verificar_firma_stripe(payload, "no-sig")
        assert ev is not None # <- esto codifica el bug
```

Estos tests asumen el path inseguro como contrato. **Hay que reescribirlos** para:

- Mantener el path permisivo solo en ENVIRONMENT=development|test (no production).
- Agregar tests especificos para ENVIRONMENT=production.

Como tests/conftest.py setea ENVIRONMENT=test por default, los tests existentes seguiran pasando si dejamos permisivo en environments no-production. Solo hace falta **agregar** los nuevos.

1.5 .env.example actual

STRIPE_WEBHOOK_SECRET **no aparece documentado**. Hay bloques para Stripe price IDs en otras zonas pero el secret no esta explicito. Hay que agregar el bloque.

1.6 Precedente util: agent/crypto.py:18-46

Patron ya implementado para ENCRYPTION_KEY:

- En production sin clave -> RuntimeError al import del modulo, aborta deploy.
- En development -> logger.warning y sigue.

Es exactamente el patron que conviene replicar para STRIPE_WEBHOOK_SECRET.

2. Plan de implementacion minimo

2.1 Cambio puntual en agent/billing.py

Agregar al **top del modulo** (igual que crypto.py):

```
_ENVIRONMENT = os.getenv("ENVIRONMENT", "development").lower()
_STRIPE_WEBHOOK_SECRET = os.getenv("STRIPE_WEBHOOK_SECRET", "")

if _ENVIRONMENT == "production" and not _STRIPE_WEBHOOK_SECRET:
    raise RuntimeError(
        "[BILLING] STRIPE_WEBHOOK_SECRET no configurada en produccion - "
        "los webhooks de Stripe podrian aceptar payloads forjados, lo que "
        "permitiria a un atacante acreditar creditos arbitrarios. "
        "Configura la variable antes de reintentar el deploy."
    )
```

Y modificar `verificar_firma_stripe` para:

- Usar `_STRIPE_WEBHOOK_SECRET` como default si no se pasa explicito.
- Si `_ENVIRONMENT == 'production'` y no hay secret -> retornar None (defensa adicional).
- Mantener path permisivo solo en non-production.

2.2 Forzar fail-fast al startup (no en el primer webhook)

Hoy *from agent.billing import ...* se hace lazy dentro del handler `webhook_stripe`. Si dejamos solo el `RuntimeError` al import del modulo, recién falla cuando llega el primer webhook Stripe (no al boot).

Para **abortar el deploy** si falta el secret:

- Opcion minima: agregar *import agent.billing # noqa: F401 # forzar check de STRIPE_WEBHOOK_SECRET al startup* cerca del top de `agent/main.py`.
- O agregar el check explicito en `lifespan()` de `agent/main.py`.

Recomendacion: opcion minima (import top-level). Es 1 linea, replica el patron de `agent.crypto` que ya se importa al top.

2.3 .env.example

```
# == Stripe webhook (acreditacion de creditos en /webhook/stripe) ==
# Obligatoria en produccion. Sin ella, el endpoint rechaza todos los
# webhooks (no acredita creditos). Obtener desde:
#   Stripe Dashboard -> Developers -> Webhooks -> tu endpoint -> Signing secret
# Empieza con "whsec_".
# STRIPE_WEBHOOK_SECRET=whsec_...
```

2.4 Total de cambio

- 1 archivo modificado en codigo (`agent/billing.py`): ~10 lineas nuevas + ajuste de `verificar_firma_stripe`.
- 1 linea agregada a `agent/main.py` (import explicito).
- 1 bloque de comentario agregado a `.env.example`.
- 1 archivo de tests modificado / extendido (`tests/test_billing.py`): ~3 tests nuevos.
- **PR pequeno, ~30-40 lineas netas.**

3. Archivos que tocaria (alcance preciso)

Archivo	Tipo de cambio
agent/billing.py	Agregar top-level: <code>_ENVIRONMENT</code> , <code>_STRIPE_WEBHOOK_SECRET</code> , <code>RuntimeError</code> si production sin secret. Modificar <code>verificar_firma_stripe</code> .
agent/main.py	Agregar <code>'import agent.billing # noqa: F401'</code> al top para forzar el check al startup.
.env.example	Agregar bloque documental para <code>STRIPE_WEBHOOK_SECRET</code> .
tests/test_billing.py	Reescribir/extender <code>TestVerificarFirma</code> con casos de <code>ENVIRONMENT=production</code> .

Lo que NO toco:

- `agent/main.py:webhook_stripe` (handler) — su logica ya rechaza con 400 si `verificar_firma_stripe` retorna `None`. No requiere cambios.
- `landing/app/api/webhook/route.ts` — independiente; ya falla con `!` si `STRIPE_WEBHOOK_SECRET` no esta. No es alcance de este PR.
- `agent/billing.py:procesar_evento_stripe` — no hace falta tocarlo.

4. Tests que correria

4.1 Tests existentes a actualizar

`tests/test_billing.py:195-207` (`TestVerificarFirma`):

- Renombrar `test_sin_secret_configurado_acepta_payload` -> `test_sin_secret_en_dev_acepta_payload`. Setear explicitamente `ENVIRONMENT=development` con `monkeypatch` para que sea robusto al default.
- Mantener `test_payload_invalidado_retorna_none` igual.

4.2 Tests nuevos a agregar

```
class TestVerificarFirmaProduction:
    def test_production_sin_secret_levanta_runtime_error_al_reload
    def test_production_con_secret_y_firma_invalida_retorna_none
    def test_production_con_secret_y_firma_valida_retorna_evento
        (mock de stripe.Webhook.construct_event)
    def test_dev_sin_secret_loguea_warning_y_acepta
        (regresion del path actual)
```

Total: 1 actualizado + 3-4 nuevos. Reload de `agent.billing` con `importlib.reload` (patron ya usado en el repo, ver `tests/test_billing.py` fixture db).

4.3 Suite completa

`pytest -q` debe quedar verde. Estimado: ~445 tests (los 419 actuales + nuevos), ~3 minutos.

5. Variables que deben estar en Render antes del merge

Solo una:

Variable	Origen	Formato esperado
STRIPE_WEBHOOK_SECRET	Stripe Dashboard -> Developers -> Webhooks -> tu endpoint /webhook/stripe -> 'Signing secret' -> 'Reveal'	empieza con 'whsec_' (~38 chars)

No solicito ver el valor. Solo confirmacion de que esta seteado en Render.

Verificacion segura del owner sin exponer el valor:

- En Render Dashboard -> tu servicio -> Environment -> confirmar visualmente que la fila STRIPE_WEBHOOK_SECRET existe.
- Opcionalmente: probar que coincide con el del Dashboard de Stripe (si Stripe lo regenero recientemente, podria haber drift).

6. Riesgos de compatibilidad y deploy

Riesgo	Probabilidad	Mitigacion
Render no tiene la env var -> deploy aborta con RuntimeError al startup	Baja (ya confirmado en J.2 del Plan v2.1 que 'se pueden setear sin downtime'; verificar antes del merge)	Setear primero, mergear despues
La env var en Render esta pero con un valor incorrecto / desactualizado	Baja-media	Verificar en Stripe Dashboard que el secret coincide con el del endpoint configurado
Hay otro endpoint Stripe en la landing usando el mismo secret pero con ruta distinta	Confirmado existente	Este PR no toca el landing. La var en Vercel debe seguir igual.
Falsos rechazos de Stripe en produccion primer dia	Baja si secret es correcto	Monitorear logs Render por '[BILLING] Firma Stripe invalida'; si aparece, validar secret
Tests existentes rompen por cambio de comportamiento	Confirmado: 1 test cambia de contrato	Plan: actualizar el test (seccion 4.1), no eliminarlo
Deploy revierte (rollback) si algo falla	Bajo	Render permite redeploy de commit anterior; el cambio es localizado a billing.py

Riesgo P0 que cierra este PR: acreditacion forjable de creditos. Eliminado en produccion.

7. Checklist antes de mergear

7.1 Pre-PR (antes de empezar)

- [] Confirmar STRIPE_WEBHOOK_SECRET esta seteada en Render con valor whsec_* correcto.
- [] Confirmar que el secret en Render coincide con el 'Signing secret' del endpoint en Stripe Dashboard (no se ha regenerado).
- [] Confirmar que la env var del landing (en Vercel, STRIPE_WEBHOOK_SECRET) sigue intacta — no la tocamos pero conviene saber que sigue ahi.
- [] Decidir nombre de branch: sugiero **pr/stripe-webhook-firma-obligatoria**.

7.2 Durante el PR (local)

- [] Implementar el cambio minimo de agent/billing.py (seccion 2.1).
- [] Agregar import top-level en agent/main.py (seccion 2.2).
- [] Agregar bloque en .env.example (seccion 2.3).
- [] Actualizar/extender tests (seccion 4).
- [] Correr *pytest -q* localmente — 100% verde.

7.3 Pre-push

- [] Verificar que el diff toca solo los 4 archivos previstos.
- [] Verificar que no se filtro el secret real en codigo, comentarios, ni tests.

[] Commit con mensaje descriptivo: *fix(seguridad): STRIPE_WEBHOOK_SECRET obligatorio en produccion (T0.2)*.

7.4 Post-push, pre-merge

[] Push a origin/pr/stripe-webhook-firma-obligatoria.

[] Abrir PR contra main en GitHub.

[] **NO mergear** hasta confirmar visualmente que la env var en Render existe.

7.5 Post-merge

[] Ver Render dashboard: deploy debe iniciar y completar sin RuntimeError al startup.

[] Si Render falla con 'RuntimeError: STRIPE_WEBHOOK_SECRET', revertir el merge inmediatamente y diagnosticar la env var antes de re-mergear.

[] Hacer una compra de prueba en Stripe (modo test) y verificar que llega el webhook y acredita correctamente.

[] Verificar logs Render por '[BILLING] Firma Stripe invalida' los primeros 30 minutos.

8. Resumen ejecutivo

Aspecto	Detalle
Riesgo que cierra	P0 — acreditacion de creditos forjable
Patron	Replica agent/crypto.py (RuntimeError en production sin secret)
Archivos	4 (1 codigo nuevo, 1 import 1 linea, 1 docs, 1 tests)
Tamano	~30-40 lineas netas
Tests	1 actualizado + 3-4 nuevos
Env var	STRIPE_WEBHOOK_SECRET debe estar en Render antes del merge
Compatibilidad	Path permisivo se mantiene en development test. Tests existentes se actualizan, no se rompen
Riesgo de deploy	Bajo si la env var esta correcta en Render
Reversible	Si, redeploy del commit anterior en Render

Estado: no implementado todavia. Esperando OK del owner para preparar el branch *pr/stripe-webhook-firma-obligatoria* con el cambio minimo descrito.